APPENDIX C

Notation and Dictionary

This appendix collects the notation of the book in one place. The symbols are grouped by subject, in roughly the order in which the book meets them: judgments and contexts, logic, type formers, the λ-calculus and evaluation, substructural logic, categories, regions and permissions, and dependent types. Each entry gives a way to read the symbol aloud and the chapter where it is introduced or explained best. The appendix ends with dictionaries that line up the same ideas in logic, type theory, category theory, Rust and Lean, first for the core of the book and then for the vocabulary of ownership and borrowing.

In this appendix you will learn:

- how to read every judgment form in the book, and which chapter defines it;
- the symbols of logic, type theory, the λ-calculus, linear logic and category theory as the book writes them;
- where two chapters use two notations for the same thing, and where one symbol does several jobs;
- the Lean spellings of the same ideas, including the book's own scoped notations and Mathlib's;
- how a single idea appears as a logical connective, a type former, a categorical construction, a Rust feature and a Lean definition.

C.1 Conventions

The letters in rules and judgments are **metavariables** (Chapter 1): placeholders for any context, term or type. The book follows the usual habits of the subject: Γ and Δ for contexts; τ, σ, ρ for types; A, B, C for propositions, or for types when the logical reading matters; e, t, u, s for terms; v, w for values; x, y, z for variables; α, β, γ for type variables. Subscripts and primes make more of them: Γ_1, e', τ_2.

Inference rules are fractions, premises above the line and the conclusion below, with the rule's name on the right in a sans-serif font. A rule with no premises is an axiom. The name usually starts with a prefix that says what kind of judgment the rule derives:

Prefix	Rules for	Ch.
T-	typing (most chapters)	1
E-	evaluation; in Chapter 14, entailment of constraints	3, 9, 14
S-	synthesis (8); splitting a context (22, 23, 28); subtyping (27)	8, 22, 27
C-	checking mode (8); the `Copy` predicate (28)	8, 28
A-	the affine λ-calculus	22
O-	outlives between lifetimes	26
R-	the region discipline (26); region constraints of NLL (29)	26, 29
K-	kinding	17
Pi-, Sigma-	formation, introduction, elimination of Π and Σ	30
∧I, $\wedge E_1$, →I, ...	introduction and elimination (natural deduction)	4
⊗R, ⊗L, ...	right and left rules (sequent calculus)	4, 21

The rule for λ-abstraction is called T-Lam in Chapters 8 to 10 and T-Abs in Chapters 12 and 13; it is the same rule.

The symbol ≈ means "corresponds to, with caveats that the text states" (Preface). When the book writes a typing rule "for Rust", it is a model of what the compiler does, not an official definition.

Symbols with more than one job

A few symbols, and a few letters, are used in several senses. The context always decides, but the list may save you a moment of confusion.

Symbol	Meanings	Ch.
$\Rightarrow$	Heyting implication; exponential object $b \Rightarrow c$	5, 19
	synthesis mode $\Gamma \vdash t \Rightarrow \tau$	8
	arms of a `case`	8, 11
	a constraint, $\pi \Rightarrow \tau$	14, 15
	a natural transformation $F \Rightarrow G$	18
$\Vdash$	forcing in a Kripke model; entailment of constraints	5, 14
$\dashv$	adjunction $L \dashv R$	19, 20
	the output context of a judgment with leftovers	22, 26
$\mapsto$	"maps to", as in $x \mapsto x^2 + 1$	3, 7
	an entry of a substitution, $[\alpha \mapsto \tau]$	13
	a store update $S[\ell \mapsto v]$; points-to $\ell \mapsto v$	28, 32
:	"has type"; between lifetimes, "outlives" ($\Delta \vdash \alpha : \beta$, `'a: 'b`)	1, 26
!	Rust's never type `!`	1, 3
	the exponential $!A$ of linear logic	21, 23
	the unique morphism $!_a$ to a terminal object	19
	"send" in a session type $!\tau.\, S$	25
?	Lean's unknowns `?m`; "why not" $?A$; "receive" $?\tau.\, S$	13, 21, 25
$*$	the kind of types; dereference $*e$; separating conjunction	17, 28, 32
$\sqcap$, $\sqcup$	meet and join of a lattice; meet of leftover contexts or of variances	5, 22, 27
$[\![\cdot]\!]$	the meaning of a syntax: in a Heyting algebra (5), as a relation (18), in a CCC (19), as a set of values (32)	5, 18, 19, 32
$\otimes$, $\multimap$	a monoidal product and its closed structure; linear logic	19, 21

Several letters change meaning between chapters too:

Letter	Meanings	Ch.
Δ	a second context; the diagonal Δ_a; a lifetime context	6, 19, 26
ρ	an environment; a type; a renaming; a region	3, 6, 9, 26
σ	a type; a substitution of de Bruijn terms; the state of the world	6, 9, 24
κ	a kind; a lifetime	17, 28
η	the η-rules; a natural transformation; the unit of a monad	8, 18, 20
π	a constraint; a projection π_1, π_2	14, 19
Σ	a Σ-type; a store typing	10, 28
S	a substitution; a session type; a store	13, 25, 28
$\mathcal{C}$	a category; the set of `Copy` base types	16, 23
$\mathcal{D}$	a derivation; a category	3, 17

Two more differences are only typographic. The opposite category is written $\mathcal{C}^{\mathrm{op}}$ in Chapters 16 and 17 and $\mathcal{C}^{op}$ in Chapters 18 and 27. The projections of a product are fst and snd in Chapter 16 and π_1 and π_2 in Chapters 19 and 30.

C.2 Judgments and contexts

Notation	Read as	Ch.
$\Gamma \vdash e : \tau$	in context Γ, the term e has type τ	1
$x_1 : \tau_1, \ldots, x_n : \tau_n$	a context: what we assume about the free variables	1
$\Gamma, x : \tau$	Γ extended with $x : \tau$	1
$(x : \tau) \in \Gamma$, $\Gamma(x) = \tau$	the context gives x the type τ	1, 3
$\vdash e : \tau$, $\varnothing \vdash e : \tau$	e has type τ in the empty context	3
$\cdot$	the empty context	22
$\dfrac{J_1 \qquad J_2}{J}$ Name	from the premises J_1 and J_2, conclude J	1
$\mathcal{D}$, $\mathcal{D}_1$	a derivation, a tree of rule instances	3
$e_1 := x$	instantiate the metavariable e_1 with x	1
$A\ true$	the judgment "A is true"	4
$\Gamma \vdash A$	sequent: assuming Γ, A is true	4
$[A]^u$	assumption A, discharged by the rule labeled u	4
$w \Vdash A$	world w forces A	5
$\Gamma \vdash t \Rightarrow \tau$	t synthesizes the type τ	8
$\Gamma \vdash t \Leftarrow \tau$	t checks against the type τ	8
$\sigma :: \Gamma$, $\Gamma(i)$	a context of de Bruijn positions, and its i-th type	9
$\lvert\Gamma\rvert \vdash \tau$	the context with variable names erased	10
Γ, α	Γ extended with the type variable α	12
$\tau \sqsubseteq \sigma$	the monotype τ is an instance of the scheme σ	13

The judgments of Parts III to VII add constraints, splitting, lifetimes, stores and semantics:

Notation	Read as	Ch.
$P \mid \Gamma \vdash e : \tau$	typing under the constraints P	14
$P \Vdash \pi$	the constraints P entail π	14
$P \Vdash d : \pi$	d is the dictionary that proves π	14
$F\ A : \kappa_2$	the type expression $F\ A$ has kind κ_2	17
$\Gamma = \Gamma_1 \uplus \Gamma_2$	Γ splits into Γ_1 and Γ_2	21, 22
$\Gamma \vdash e : \tau \dashv \Delta$	e has type τ and leaves Δ unused	22
$\mathsf{Copy}(\tau)$	τ is a `Copy` type	23, 28
$\Delta; \Gamma \vdash c \dashv \Delta'; \Gamma'$	the command c turns regions Δ and context Γ into Δ' and Γ'	26
$\Delta \vdash \alpha : \beta$	under Δ, the lifetime α outlives β	26
$\sigma <: \tau$	σ is a subtype of τ	27
$\Delta; \Gamma \vdash e : \tau$	typing with the lifetimes Δ in scope	28
$\cdot; \cdot \underset{\Sigma}{\vdash} e : \tau$	a run-time term, typed with the store typing Σ	28
$\Gamma \vDash e : \tau$	e is semantically well typed	32
$\{P\}\ c\ \{Q\}$	from a heap satisfying P, c runs safely to one satisfying Q	32

The turnstile $\vdash$ is a symbol of the metalanguage. It separates assumptions from a conclusion and is never part of a term, which is the difference between $A \vdash B$ (a judgment) and $\vdash A \to B$ (a judgment about the proposition $A \to B$); Chapter 4 explains why the two are derivable together. In Lean's infoview the same symbol `⊢` separates the hypotheses from the goal (Chapter 2).

C.3 Logic

Notation	Read as	Ch.
$A \wedge B$, $A \vee B$	A and B; A or B	4
$A \to B$	A implies B	4
$\top$, $\bot$	true; false	4
$\neg A$	not A, defined as $A \to \bot$	4
$A \leftrightarrow B$	A if and only if B, defined as $(A \to B) \wedge (B \to A)$	4
$\forall x.\, A(x)$, $\exists x.\, A(x)$	for all x; there exists an x	5, 10
$\wedge$I, $\wedge$E$_1$, $\wedge$E$_2$	introduction and eliminations of $\wedge$	4
$\vee$I$_1$, $\vee$I$_2$, $\vee$E	introductions and elimination of $\vee$	4
$\to$I, $\to$E	introduction (discharging) and elimination (*modus ponens*)	4
$\top$I, $\bot$E	truth; *ex falso quodlibet*	4
Hyp	use an assumption from the context	4
Cut	prove a lemma, then use it	4, 21
RAA	*reductio ad absurdum*, the classical rule	4
$A \mathit{tonk} B$	Prior's connective without harmony	4
$\mathcal{D}_1[\mathcal{D}_2/A]$	the derivation $\mathcal{D}_2$ substituted for the assumption A	4
Exch, Weak, Contr	the structural rules: exchange, weakening, contraction	6
$w \le w'$	w' is a possible future of world w	5
$w \Vdash A$, $w \nVdash A$	w forces A; w does not force A	5
$a \sqcap b$, $a \sqcup b$	meet ("and") and join ("or") of truth values	5
$a \Rightarrow b$	Heyting implication, characterized by residuation	5
$c \le (a \Rightarrow b)$ iff $c \sqcap a \le b$	residuation	5
$[\![\Gamma]\!] \le [\![A]\!]$	the sequent is valid in a Heyting algebra	5

In Lean, the connectives on propositions are `∧`, `∨`, `→`, `¬`, `↔`, `True`, `False`, `∀ x, P x` and `∃ x, P x` (Chapters 4 and 10), and Mathlib writes the Heyting operations as `⊓`, `⊔`, `⇨`, `⊤` and `⊥` (Chapter 5).

C.4 Type formers

Notation	Read as	Ch.
$\mathbf{1}$	the unit type, with one value; Rust `()`	1, 11
$\mathbf{0}$	the empty type, with no values; Rust `!`, `Infallible`	1, 11
$A \times B$	product: a pair of an A and a B	8, 11
$A + B$	sum: an A or a B, tagged	8, 11
$A \to B$, B^A	function type, also called exponential	8, 11
$\lvert A \rvert$	the number of values of A	11
$A \cong B$	A and B are isomorphic	11
$\mu X.\, F(X)$	the recursive type X with $X \cong F(X)$	11
∂F	the derivative of a type: F with a hole	11
$\forall \alpha.\, \tau$	for every type α, τ (System F)	12
$\sigma ::= \tau \mid \forall \alpha.\, \sigma$	a type scheme: quantifiers only at the outside	13
$\pi \Rightarrow \tau$	τ qualified by the constraint π	14
$\exists \alpha.\, \tau$	for some hidden type α, τ	15
$\Pi x : A.\, B$	dependent function type	10, 30
$\Sigma x : A.\, B$	dependent pair type	10, 30
$\mathrm{List}\, A$, $\mathrm{Vec}\, \alpha$	type constructors applied to a type	11, 12
$F : \kappa_1 \to \kappa_2$	F takes a type of kind κ_1 to one of kind κ_2	17
$M\, \tau$	a computation with effect M that produces a τ	20
Const_A	the constant functor at A	18
$!\tau.\, S$, $?\, \tau.\, S$, end	send a τ, receive a τ, stop (session types)	25
$\overline{S}$	the dual of the session type S	25
$\delta(s, a) = s'$	in state s, action a leads to s'	25
$\mathit{Box}\, \tau$, $\mathit{Ref}\, \tau$	an owning pointer; a reference	26
$\&^{\kappa}\, \tau$, $\&^{\kappa}\, \mathit{mut}\, \tau$	shared and mutable references with lifetime κ	28

The connectives of linear logic, which are also type formers of the affine λ-calculus, are in the section on substructural logic below.

C.5 The λ-calculus and evaluation

Notation	Read as	Ch.
$\lambda x.\, t$	the function with parameter x and body t	7
$t\ u$	t applied to u (application associates to the left)	7
$\lambda x : \tau.\, t$	a λ with a type annotation (Church style)	8
$\mathrm{FV}(t)$	the free variables of t	7
$t \underset{\alpha}{\equiv} u$	t and u differ only in bound names	7
$t[x := s]$	t with s substituted for x	7
$t[s/x]$	the same operation, in the other common notation	4, 6
$\lambda.\, t$, 0, 1	a nameless binder and de Bruijn indices	7
$\#i$, $\lambda : \tau.\, t$	de Bruijn variables and binders of Chapter 9	9
shift_c, shift_c^d	add 1 (or d) to the indices at least c	7
$t[j := s]$, $t[v]$	substitution for an index; for index 0	7, 9
$(\lambda x.\, t)\, s$	a redex	7
$t \to t'$	t takes one step to t'	1, 7
$t \xrightarrow{*} t'$	t reaches t' in zero or more steps	7
$\rho \vdash e \Downarrow v$	under environment ρ, e evaluates to v (big-step)	3
$\rho, x \mapsto v$	the environment ρ extended with x bound to v	3
Ω	$(\lambda x.\, x\, x)\, (\lambda x.\, x\, x)$, which steps to itself	7
Y, Z	fixed-point combinators (lazy; call-by-value)	7
K, I	$\lambda x.\, \lambda y.\, x$ and $\lambda x.\, x$	7
$\overline{n}$	the Church numeral n	7
$\lambda x : \sigma.\, f\, x \equiv f$	the η-rule for functions	8
`fix` f, $\mathit{fix}\, f.\, e$	general recursion, which a terminating calculus lacks	8, 22

For polymorphism, existential types and type inference (Chapters 12 to 15) and for the resource languages of Part V:

Notation	Read as	Ch.
$\Lambda\alpha.\, e$, $e\,[\tau]$	type abstraction; type application	12
$\tau[\sigma/\alpha]$	τ with σ for the type variable α	12
$\tau[\alpha := \sigma]$	the same, in the other notation	14, 15
$\mathit{pack}\ \langle \sigma, e\rangle\ \mathit{as}\ \exists\alpha.\,\tau$	an existential package	15
$\mathit{unpack}\ \langle \alpha, x\rangle = e_1\ \mathit{in}\ e_2$	open a package	15
$[\alpha \mapsto \tau]$, $S\,\tau$	a substitution; applying it to τ	13
$S_2 \circ S_1$	first S_1, then S_2	13
$\tau_1 \doteq \tau_2$	an equation to be solved by unification	13
$\mathrm{ftv}(\tau)$, $\mathrm{Gen}(\Gamma, \tau)$	free type variables; generalization	13
mgu, $W(\Gamma, e) = (S, \tau)$	most general unifier; algorithm W	13
$\langle \sigma, e\rangle \to \langle \sigma', e'\rangle$	a step that also changes the state σ	24
$\mathit{release}_\tau$	the destructor of τ	24

Two notations for substitution. Chapters 4 and 6 write $t[s/x]$ ("t with s for x"), and Chapter 7 introduces $t[x := s]$ for the same operation, noting that both are common. After that the book uses both: $t[x := s]$ in Chapters 7, 8 and 15, and $t[s/x]$ in Chapters 12, 19, 22, 23, 28 and 30. For type variables, Chapters 12 and 18 write $\tau[\sigma/\alpha]$ and Chapters 14 and 15 write $\tau[\alpha := \sigma]$. Chapter 13 writes a substitution as a map $[\alpha \mapsto \tau]$ applied on the left, $S\,\tau$. Chapters 9, 10 and 19 also write $t[s]$ for substituting s for the variable of the innermost λ. In Chapter 26, $s[env(y) := \ldots]$ is not a substitution but an update of the state s.

C.6 Substructural logic

Notation	Read as	Ch.
Exch, Weak, Contr	reorder, discard, duplicate an assumption	6
$A \otimes B$	"A tensor B": both, from split resources	6, 21
$A \,\&\, B$	"A with B": either one, chosen by the user	6, 21
$A \oplus B$	"A plus B": one of them, chosen by the producer	21
$A \parr B$	"A par B": the classical multiplicative disjunction	21
$A \multimap B$	"A lollipop B": consume an A, produce a B	8, 21
1, $\top$, $\bot$, 0	the units of $\otimes$, $\&$, $\parr$ and $\oplus$	21
$A^\bot$	the linear negation of A	21
$!A$, $?\,A$	"of course A", "why not A": the exponentials	21, 23
$!\Gamma$	a context in which every formula has the form $!A$	21, 23
$\Gamma_1 \uplus \Gamma_2$	the disjoint union of two contexts	21, 22
Ax, Cut	axiom $A \vdash A$; cut	21
$\otimes$R, $\otimes$L, &R, &L$_1$, $\oplus$R$_1$, $\multimap$R, $\multimap$L	right and left rules of the connectives	21
Der, !W, !C, Prom	the rules of ! (Chapter 21's names)	21
Dereliction, !-Weak, !-Contr, Promotion	the same rules (Chapter 23's names)	23

The affine λ-calculus of Chapters 22 and 23 writes the same connectives as types, with these term formers and rules:

Notation	Read as	Ch.
λ^{aff}	the affine λ-calculus	22
(e_1, e_2), $\texttt{let}\ (x, y) = e_1\ \texttt{in}\ e_2$	a $\otimes$ pair and its eliminator	22
$\langle e_1, e_2 \rangle$, $\mathit{fst}\ e$, $\mathit{snd}\ e$	an offer (a $\&$ pair) and its projections	22
$\texttt{case}\ e\ \{x.\, e_1 \mid y.\, e_2\}$	case analysis on a $\oplus$	22
$\texttt{drop}\ x; e$	discard x, then continue with e	22, 24
S-Empty, S-Left, S-Right	the rules of context splitting	22
S-Both	a `Copy` variable may go to both sides	23
$\Gamma - x$	Γ without x	22
$\Delta_1 \sqcap \Delta_2$	the variables left unused by both alternatives	22
$\mathit{derelict}\ e$, $!e$	use a $!\tau$ once; build a $!\tau$	23
$\mathit{dup}_\tau : \tau \multimap \tau \otimes \tau$	an explicit duplication, like `Clone::clone`	23
$P * Q$	separating conjunction: P and Q on disjoint resources	26, 32

Linear logic writes the two conjunctions as $\otimes$ and $\&$, and the book follows it. In Rust, $A \otimes B$ is `(A, B)` and $A \multimap B$ is `FnOnce(A) -> B`; the full correspondence is in the second dictionary at the end of this

appendix. The book's Lean files declare the same connectives as scoped notation for their own formula and type syntax: `⊗`, `&`, `⊕` and `⊸` (Chapters 21 and 22).

C.7 Categories

Notation	Read as	Ch.
$\mathcal{C}$, $\mathcal{D}$	categories	16, 17
$\mathrm{Hom}(a, b)$	the morphisms from a to b	16
$f : a \to b$	f is a morphism from a to b	16
id_a	the identity morphism on a	16
$g \circ f$	g after f: first f, then g	16
$a \cong b$	a and b are isomorphic	16
$\star$	the single object of a monoid seen as a category	16
$a \times b$; $\langle f, g \rangle$	a product; the pairing of f and g	16, 19
fst, snd; π_1, π_2	the projections of a product	16, 19
inl, inr; $[f, g]$	the injections of a coproduct; copairing	16
$\mathcal{C}^{\mathrm{op}}$	the opposite category: every arrow reversed	16
$F : \mathcal{C} \to \mathcal{D}$	a functor	17
$F\,a$, $F\,f$	the functor on an object; on a morphism	17
$\mathcal{C} \times \mathcal{D}$	the product of two categories	17
$\mathbf{Cat}$	the category of categories and functors	17

Natural transformations, cartesian closed categories, monoidal categories and monads (Chapters 18 to 20, and 27):

Notation	Read as	Ch.
$\eta : F \Rightarrow G$, η_a	a natural transformation; its component at a	18
$\theta_a \circ \eta_a$	vertical composition, componentwise	18
$[\mathcal{C}, \mathcal{D}]$, $\mathcal{D}^{\mathcal{C}}$	the functor category	18
$\mathrm{Hom}(\alpha, -)$	the functor of morphisms out of α	18
$\mathrm{Nat}(F, G)$	the natural transformations from F to G	18
$[\![\tau]\!]_R$	the relation that τ assigns to a relation R	18
$!_a$	the unique morphism from a to the terminal object	19
$f \times g$	f and g side by side, $\langle f \circ \pi_1, g \circ \pi_2 \rangle$	19
$\Delta_a = \langle \mathrm{id}_a, \mathrm{id}_a \rangle$	the diagonal $a \to a \times a$	16, 19
$b \Rightarrow c$, c^b	the exponential object	19
eval, $\mathrm{curry}\ f$	evaluation; the curried form of f	19
$L \dashv R$	L is left adjoint to R	19
$(-) \times b$	the functor "pair with a b"	19
$[\![\Gamma \vdash t : \tau]\!]$	the morphism that interprets a typed term	19
$a \otimes b$, I	a monoidal product and its unit	19
$b \multimap c$	the exponential of a monoidal closed category	19
pure, *bind*	the operations of a monad	20
$g \circ_M f$	Kleisli composition	20
η, μ	unit and multiplication (`join`) of a monad	20
$\mathcal{L}$, $\mathcal{T}$	the categories of lifetimes and of types	27

The book composes from right to left, $g \circ f$, and so does its own Lean notation `g ∘ f`; Mathlib composes in diagrammatic order, `f ≫ g`. The section on Lean notation below lists both libraries' symbols.

C.8 Regions, lifetimes and permissions

Notation	Read as	Ch.
`'a`, `&'a T`, `&'a mut T`	a lifetime; references valid during `'a`	26
`'a: 'b`	`'a` outlives `'b`	26
`T: 'a`	every reference inside `T` outlives `'a`	26
`'static`, *static*	the region of the whole execution	26
`for<'a>`	for every lifetime `'a` (a higher-ranked bound)	12
`letregion` ρ `in` e	a region that lives while e runs	26
ref ρ	a pointer into region ρ	26
$\rho \in \Delta$	the region ρ is still live (escape check)	26
ℓ	a location in memory	26, 28
O-Refl, O-Hyp, O-Static, O-Trans	the rules of outlives	26
$\& \alpha\ \tau <: \& \beta\ \tau$	a reference may be used for a shorter lifetime	27

The core calculus of borrowing of Chapter 28:

Notation	Read as	Ch.
borrow $y = \&^{\alpha} x$ *in* e	borrow x as y for the lifetime α, during e	28
borrow $y = \&^{\alpha}$ *mut* $* e_1$ *in* e_2	a reborrow	28
$* e$, $* e_1 := e_2$	read through a reference; write through `&mut`	28
$\mathrm{flv}(\tau)$	the lifetime variables free in τ	28
$\alpha \notin \mathrm{flv}(\sigma)$	the escape check of the borrow calculus	28
own ℓ, *shr* ℓ, *uniq* ℓ	run-time owner, shared and unique references	28
$S(\ell) = v^m$	location ℓ holds v, with loan state m	28
none, *shared*, *mut*	not lent; lent shared; lent mutably	28
$\langle S; e \rangle$	a configuration: store and term	28
restore w *after* e	give the lent value back when e finishes	28
$\mathrm{dom}(S)$, $\mathrm{free}(S, v)$	allocated locations; release what v owns	28

Non-lexical lifetimes and Polonius (Chapter 29), and the separation logic and lifetime logic of Chapter 32:

Notation	Read as	Ch.
$P \in \rho$	the program point P belongs to the region ρ (NLL)	29
$\rho_1 : \rho_2$	region ρ_1 outlives ρ_2 (contains its points)	29
`subset('a, 'b)`, `loan_live_at(L, P)`	Polonius relations over origins, loans and points	29
$\ell \mapsto v$	points-to: the cell ℓ holds v	32
$\{P\}\ c\ \{Q\}$, Frame	Hoare triple; the frame rule	32
$[\dagger\ \kappa]$	the token saying that lifetime κ has ended	32
$\&^{\kappa}_{\text{full}} P$	a full borrow of P for the lifetime κ	32
$\triangleright$	"later", a modality of Iris	32

Chapters 26 to 28 write lifetimes as Greek letters, α, β, γ and, in Chapter 28, κ, where Rust writes `'a` and `'b`. The regions of Chapter 26's heap language, after Tofte and Talpin, and the regions of NLL in Chapter 29 are written ρ.

C.9 Dependent types

Notation	Read as	Ch.
$\Pi x : A.\ B$	for every $x : A$, a B (which may mention x)	10, 30
$\Sigma x : A.\ B$	an $x : A$ together with a B for that x	10, 30
$B[a/x]$	the type B at the argument a	30
$\pi_1\ p$, $\pi_2\ p$	the components of a dependent pair	30
Pi-Form, Pi-Intro, Pi-Elim	the rules of Π-types	30
Sigma-Form, Sigma-Intro, Sigma-Fst, Sigma-Snd	the rules of Σ-types	30
Type, *Sort* u, imax $u\ v$	universes, and the level of a Π-type	12, 30
$\lambda\!\!\rightarrow$, $\lambda 2$, $\lambda\omega$, $\lambda\underline{\omega}$, λP, λC	the corners of the λ-cube	30

In Lean, $\Pi x : A.\ B$ is written `(x : A) → B` (Chapter 12) and becomes the ordinary arrow `A → B` when B does not mention x. $\Sigma x : A.\ B$ is `(x : A) × B` or `Σ x, B` for data, `(x : A) ×' B` when the second component may be a proof, and `∃ x, B` for propositions (Chapters 10, 15 and 30). Equality is the type `a = b`, proved by `rfl`, and `h ▸ e` transports `e` along `h : a = b` (Chapter 30). The subtype `{x // p x}` pairs a value with a proof (Chapters 10 and 11). `Fin n` is the type of natural numbers below `n` (Chapter 30), and `xs[i]'h` indexes with a proof `h` that `i` is in range (Chapter 1).

C.10 Lean notation

Most of Lean's notation is explained in Chapter 2 and in Appendix B. This table lists the symbols that the chapters use, including the scoped notations that the book's own Lean files declare.

Lean	Meaning	Ch.
`fun x => e`, `(· + 1)`	anonymous functions; `·` is the argument	1, 2
`⟨a, b⟩`	anonymous constructor: a pair, a proof of `∧`, a structure	1
`{α : Type}`, `[C α]`	implicit argument; instance argument	2
`@f`	`f` with all its arguments explicit	12
`⊢`	in the infoview: the goal, after the hypotheses	2
`·` (in a proof)	focus on one goal	2
`t₁ <;> t₂`	run `t₂` on every goal that `t₁` produces	2
`←`	bind in `do` notation; rewrite right to left in `rw`	2
`↑x`	a coercion inserted by Lean	27
`ℕ`, `ℤ`, `ℕ∞`	naturals, integers, naturals with a top element `⊤`	1, 23
`Prop`, `Type`, `Type u`, `Sort u`	the universes	10, 12
`α ⊕ β`, `.inl`, `.inr`	the sum type	10, 11
`?m.5`	a metavariable: an unknown the elaborator must solve	13
`a ⋄ b`	the book's monoid operation	14
`<$>`, `>>=`	`Functor.map`; `bind`	17, 20
`g ∘ f`	function composition	16
`α ≅ β`	the book's `Iso` (in Chapter 16, between objects)	11, 16
`α ≃ β`	Mathlib's `Equiv`: a bijection with both proofs	11
`a ⟶ b`, `g ⊚ f`, `id a`	the book's morphisms, composition, identity	16
`X ⟶ Y`, `f ≫ g`, `𝟙 X`	Mathlib's morphisms, composition (first `f`), identity	16
`C ⥤ D`, `F ⋙ G`, `𝟭 C`	Mathlib's functors, their composition, the identity functor	17
`a × b`, `b ⟹ c`	the book's product and exponential objects	19
`σ ⇒ τ`	the function type of a toy language	8, 13, 27
`σ <: τ`	subtyping in the toy language of Chapter 27	27
`P ∗ Q`, `ℓ ↦ v`, `P ⊢ Q`	separation logic: conjunction, points-to, entailment	32

The commands `#eval`, `#check`, `#guard_msgs` and `#print axioms` are introduced in Chapter 1 and `#print` in Chapter 4; the tactics are described in Appendix D.

C.11 Dictionary: logic, types, categories, Rust, Lean

The table extends the dictionaries of Chapters 10 and 19. A Rust entry is the closest counterpart, with the caveats the chapters state; "—" means that the book gives no counterpart.

Logic	Type theory	Categories	Rust	Lean
proposition	type	object	type	`p : Prop`, `α : Type`
proof	term	morphism	value, function	proof term
assumption	variable in the context	projection out of $[\![\Gamma]\!]$	parameter	hypothesis `h : p`
$A \wedge B$	$A \times B$	product	`(A, B)`, a struct	`p ∧ q`, `α × β`
$A \vee B$	$A + B$	coproduct	an `enum`, `Result`	`p ∨ q`, `α ⊕ β`
$A \to B$	$A \to B$	exponential B^A	`fn`, closures	`p → q`, `α → β`
$\top$	$\mathbf{1}$	terminal object	`()`	`True`, `Unit`
$\bot$	$\mathbf{0}$	initial object	`!`, `enum Void {}`	`False`, `Empty`
$\neg A$	$A \to \mathbf{0}$	arrow into the initial object	`fn(A) -> Void`	`¬p`
$\forall x.\, P(x)$	Π-type	—	generics, over types only	`∀ x, P x`
$\exists x.\, P(x)$	Σ-type	—	`impl Trait`, over types only	`∃ x, P x`
→I	λ-abstraction	curry	closure	`fun`, `intro`
→E	application	eval	call	application, `apply`
⊥E	`absurd`, empty `match`	arrow out of the initial object	`match x {}`, `!` coerces	`nomatch`, `absurd`
replacing an assumption by its proof	substitution $t[u/x]$	composition	inlining a call	—
detour	redex	—	—	—
normalization	β-reduction	$\mathrm{eval} \circ (\mathrm{curry}\, f \times \mathrm{id}) = f$	running a program	`rfl`
local completeness	η	uniqueness of curry	`.map(f)` for `.map(|x| f(x))`	`rfl` (definitional η)
contraction	using x twice	diagonal Δ	needs `Copy` or `Clone`	free
weakening	unused variable	$!_a : a \to 1$	dropping a value	free
consistency	no closed term of type $\mathbf{0}$	—	fails: `loop {}`, `panic!`	holds: totality
induction	recursor, fold	initial algebra	recursion with `match`	`induction`, `T.rec`
intuitionistic $\wedge$, $\to$, $\top$	simply typed λ-calculus	cartesian closed category	—	proofs without `Classical`
Heyting algebra	—	thin bicartesian closed category	—	`HeytingAlgebra`
excluded middle	—	—	—	`Classical.em`

The Rust column for $A \to B$ depends on the reading. Chapter 10 pairs it with `impl FnOnce(A) -> B`, a function that may be called once, which is enough to read a proof; Chapter 19, which needs functions that can be used freely, pairs the exponential with `Box<dyn Fn(A) -> B>`. In the linear reading of Part V the same `FnOnce` is $A \multimap B$.

A second table covers the abstraction mechanisms of Parts III and IV.

Type theory	Logic or categories	Rust	Lean	Ch.
$\forall \alpha.\, \tau$	$\forall$ over propositions (second order)	generic function	`(α : Type) → τ`	12
type constructor	functor on objects	`Option`, `Vec`	`Option : Type → Type`	17
`map` with its laws	functor on morphisms	`map` methods, no trait	`Functor`, `LawfulFunctor`	17
polymorphic $\forall \alpha.\, F\,\alpha \to G\,\alpha$	natural transformation	`fn f<T>(Vec<T>) -> Option<T>`	`NatTrans`	18
parametricity	naturality, free theorems	leaks: `Any`, `size_of`	not provable inside Lean	18
type class	a predicate on types	trait	`class`	14
instance	an entailment rule, a Horn clause	`impl`	`instance`	14
dictionary	evidence for a constraint	vtable, or monomorphized code	instance argument `[C α]`	14
$\exists \alpha.\, \tau$	$\exists$ over types	`impl Trait`, `dyn Trait`	a Σ-type over `Type`	15
$M\ A$ with *pure*, *bind*	monad, Kleisli category	`Option`, `Result`, `?`	`Monad`, `do`	20
currying	adjunction $(-) \times b \dashv b \Rightarrow (-)$	closures returning closures	`fun a b => ...`	19
subtyping $\sigma <: \tau$	a preorder, a thin category	lifetimes only	none (coercions `↑`)	27
variance	covariant or contravariant functor	`&'a T`, `fn(T)`, `&mut T`	—	17, 27

C.12 Dictionary: resources, ownership and borrowing

The vocabulary of Parts V and VI lines up as follows. The first table is about the structural rules, the second about the connectives of linear logic, and the third about references and lifetimes.

Structural rule	On resources	Categories	Rust	Ch.
exchange	use in any order	symmetry $a \otimes b \cong b \otimes a$	order of `let`s is irrelevant to typing	6, 19
weakening	discard	a map $a \to 1$	drop; runs `Drop`	6, 24
contraction	duplicate	diagonal $a \to a \times a$	`Copy` (implicit), `Clone` (explicit)	6, 23
no contraction	use at most once (affine)	symmetric monoidal, terminal unit	moves; E0382	19, 22
no weakening	use at least once (relevant)	—	`#[must_use]`, a lint	21, 24
neither	use exactly once (linear)	symmetric monoidal closed	not enforced: `mem::forget` is safe	19, 21, 24

Linear logic	Name	λ^{aff}	Rust ≈	Ch.
$A \otimes B$	tensor	pair, `let (x, y)`	`(A, B)`, a struct	21, 22
$A \,\&\, B$	with	offer $\langle e_1, e_2 \rangle$	a value with two consuming methods	21, 22
$A \oplus B$	plus	`inl`, `inr`, `case`	an `enum`	21, 22
$A \parr B$	par	—	— (classical only)	21
$A \multimap B$	lollipop	$\lambda x : \tau . e$	`FnOnce(A) -> B`	21, 22
1	unit of $\otimes$	$()$	`()`	21, 22
$!A$	of course	`Copy` predicate, S-Both	`A: Copy`	21, 23
Promotion	—	`Copy` only from `Copy` parts	`#[derive(Copy)]` needs `Copy` fields (E0204)	23
!-Weak	—	discarding a `Copy` value is free	`Copy` excludes `Drop` (E0184)	23
$\tau \multimap \tau \otimes \tau$	a term, not a rule	dup_τ	`Clone::clone`	23
—	counted sharing	—	`Rc<T>`, `Arc<T>`	23

Theory	Rust	Ch.
multiplicity: how many times a variable is used	moves, `Copy`	23, 26
extent: where a reference is valid	lifetimes	26
region variable	lifetime parameter `'a`	26
outlives constraint	`'a: 'b`, `T: 'a`	26
escape check, $\rho \in \Delta$ or $\alpha \notin \mathrm{flv}(\sigma)$	E0597, E0515	26, 28
duplicable read permission	`&T`, which is `Copy`	26
unique read-write permission	`&mut T`, which is affine	26
temporary transfer of permission	a borrow	26, 28
a new unique reference for a shorter lifetime	a reborrow `&mut *r`	28
aliasing XOR mutation	E0499, E0502, E0506	26, 29
run-time check of the aliasing rule	`Cell`, `RefCell`	26
region as a set of program points	non-lexical lifetimes	29
loan	a borrow expression in MIR	29
origin: a set of loans	Polonius	29
variance as functoriality	`&'a T` covariant, `&mut T` invariant in `T`	27
typestate: states as types	consuming methods, `PhantomData`	25
separating conjunction $P * Q$	two live `&mut` are disjoint	32
proof obligation	an `unsafe` block, a `SAFETY:` comment	32